Real-world (part 2) — run in parallel, fall back, and stream — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — Three LCEL tools, one idea each
- Hint 2 — The plan, and almost the whole thing

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — Three LCEL tools, one idea each

Basic Version

Every chain is a `Runnable`, so three extra tools come for free (the LCEL topic):

- **Run two things at the same time:** `RunnableParallel({"weather": get_weather, "news": get_news})`. Each function gets the same input; the result is a dict with the same keys.
- **A backup when a call fails:** `chain.with_fallbacks([backup])`. `RunnableLambda(backup_reply)` turns a plain function into a step it accepts.
- **The reply in pieces:** `for chunk in chain.stream(inputs):` — the same idea as Doc04's streaming exercise, through LCEL.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Make each one provable:

- Time the parallel step with `time.time()` before and after. Two functions that each `time.sleep(1)` should take about 1 second together, not 2.
- Fall back only on errors that pass on their own: `exceptions_to_handle=(openai.APIConnectionError, openai.RateLimitError)`. A `KeyError` from your own code must still raise.
- Force a real outage with no key and no cost: `ChatOpenAI(model="gpt-4o-mini", base_url="http://127.0.0.1:9", max_retries=0)` — nothing listens on port 9.
- Join the streamed pieces into one string, so the caller gets the full text too.

Difference between Basic and Intermediate: Basic names the three tools. Intermediate shows how to prove each one works — a timing, a forced outage, a bug that must not be hidden, and the joined text.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
lookups = RunnableParallel(weather=..., news=...)
time it, print the sorted keys

broken chain = prompt | model on port 9 | parser
safe chain = broken.with_fallbacks([RunnableLambda(backup_reply)], ...)
print what safe chain returns

for chunk in live chain.stream(...): print it, no newline
```

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
build_lookup_step() -> Runnable        # the RunnableParallel
with_backup(chain) -> Runnable         # .with_fallbacks(...)
stream_text(chain, inputs) -> str      # print pieces, return the joined text

main:
    case 1: keys, and "under 1.5 s?"
    case 2: outage -> backup used
    case 3: a buggy step -> KeyError still raised
    case 4: stream, then the full text's length
```

Write it, run it, then compare against the Solution.

Difference between Basic and Intermediate: Basic runs each tool once. Intermediate wraps each in a named function and checks it with a `case` line — the shape Project 6 builds on.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution